

5.13

1.CLIP 创新工作

1.1 简介

CLIP (Contrastive Language-Image Pre-training) 是由 OpenAI 在2021年推出的一个极具开创性的多模态模型，它通过学习将图像和文本映射到同一个语义空间，成功打破了视觉和自然语言之间的障碍。

1.2 自然语言监督

概括一下自然语言监督，就是用文本 (句子、段落) 代替固定的标签。

表1 自然语言监督与传统监督学习的区别

维度	传统监督学习	自然语言监督
监督信号	独热编码的类别 ID（硬标签）	句子、短语等自然语言描述（软标签）
输出空间	封闭、固定的类别集合	开放、无限的文本语义空间
泛化能力	差，难以识别训练集外的概念	强，可以利用语言的知识做零样本推理
标注成本与噪声	人力成本高，但可用标签相对精确	数据易得（网络图文），但噪声大、歧义多
语义粒度	粗粒度，无法区分属性和关系	细粒度，可描述颜色、动作、场景、风格等

- 在 CLIP 诞生之前，在自然语言监督下进行学习不是一个新方法，以前有相关工作提到过，但这些工作都没有做到很大规模，得到很好的效果。
- 采用自然语言监督，不需要对数据进行标注。
- 与自然语言结合后，不单单是视觉特征，而是多模态的特征，这有利于 zero-shot 迁移。

这样训练之后的模型是多模态模型，让大家看到多模态模型的巨大潜力。

1.3 大规模数据集

已有的数据集要么数据量不够多，要么标注质量很差。

于是作者团队从互联网上大规模采集数据，构造了4亿图像-文本对，构成了 WIT (WebImageText) 数据集。

贡献了一个很大的数据集，“大力出奇迹”。

1.4 对比学习

为了学习到开放世界的众多视觉概念，计划在 WIT 这个规模如此庞大的数据集上进行训练，因此训练效率至关重要。如果采用之前的一些学习方法，那么训练所花费的资源非常巨大，即便是 OpenAI 这样的公司都无法承受。

 我们来举个例子：

假如我们像训练 GPT 一样训练现在这个模型，输入一张图片，我们去预测这张图片对应的文本。

首先图片的信息量就要比纯文本大，训练资源本身就消耗很大，其次可以设想一下，一张图片被描述成一段文本其实有很多种形式，这无疑加大了训练难度与预测难度。

CLIP 采用了对比学习。

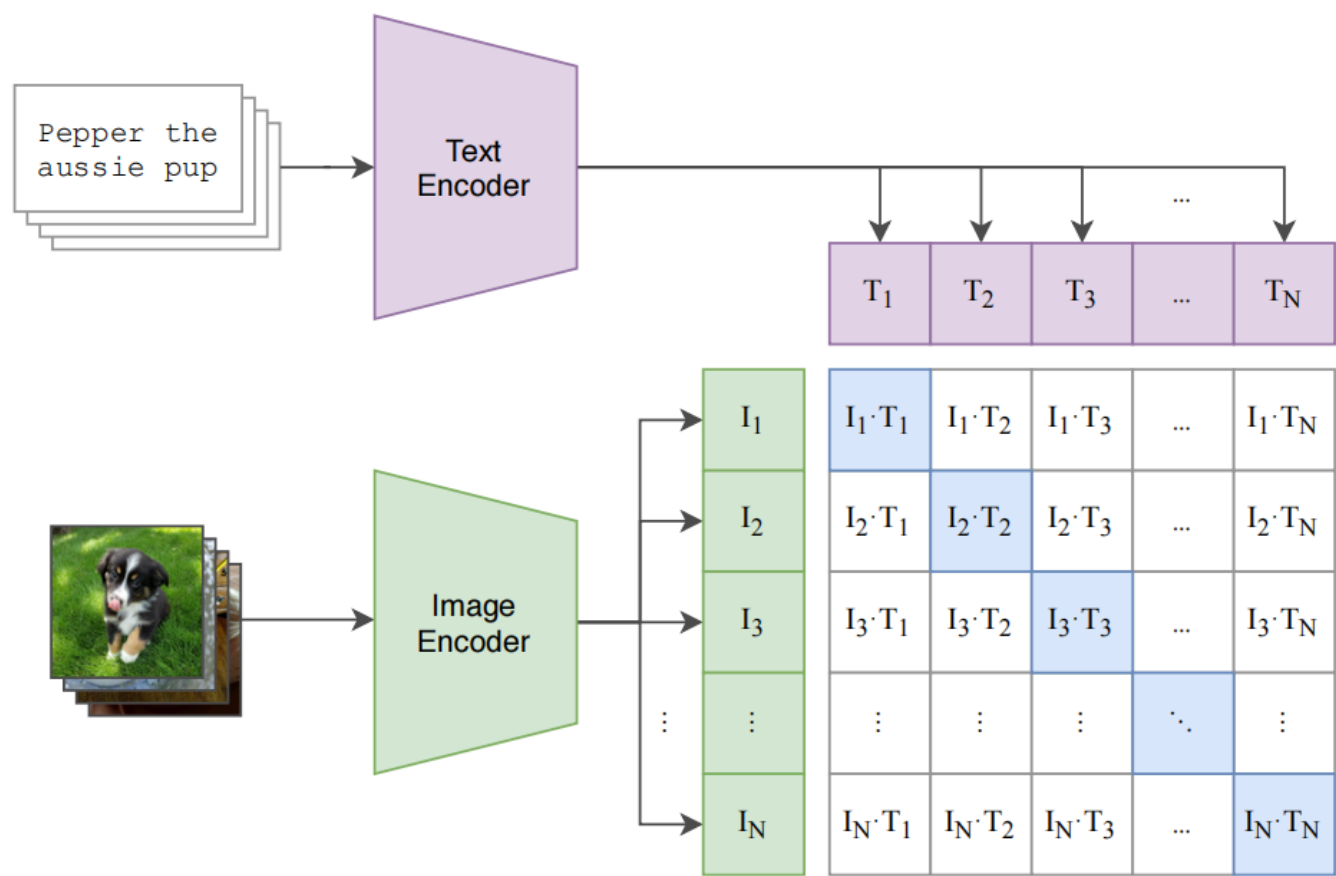


图1.1 预训练示意图

预训练时候的任务简单概括一下就是让模型判断一对图文是否匹配。

1. 分别编码图文，把他们变成向量。

2. 计算 N 阶矩阵的余弦相似度，在这 N 对数据中，位于对角线上的图文向量是匹配的，其他的是不匹配的。

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

图1.2 余弦相似度计算公式

3. 计算损失，采用了对称的交叉熵损失。简而言之，对于给定文本，计算和所有图像的相似度，对于给定图像，计算和所有文本的相似度，两种加起来取平均，可以看作是对整个矩阵对应的元素都求过损失。

$$\mathcal{L}_{\text{img} \rightarrow \text{txt}} = -\frac{1}{N} \sum_{i=1}^N \log \left(\frac{e^{s_{ii}}}{\sum_{j=1}^N e^{s_{ij}}} \right)$$

$$\mathcal{L}_{\text{txt} \rightarrow \text{img}} = -\frac{1}{N} \sum_{j=1}^N \log \left(\frac{e^{s_{jj}}}{\sum_{i=1}^N e^{s_{ij}}} \right)$$

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2} (\mathcal{L}_{\text{img} \rightarrow \text{txt}} + \mathcal{L}_{\text{txt} \rightarrow \text{img}})$$

图1.3 损失的计算



温度参数，控制 softmax 的“尖锐度”：越小，分布越尖锐，模型对负样本的惩罚越严厉；越大，分布越平滑。

$$p_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}$$

图1.4 引入温度参数的 softmax

🌟 Image Encoder 部分，原论文尝试过很多，比如 ResNet、ViT，最终选择了效果最好的一个 ViT。

1.5 Prompt Template

这一部分创新工作我也认为比较巧妙。

CLIP 在预训练的时候，见到的是很多文本，但是做分类任务的时候，很多类别标签只有一个词，这多多少少有点不匹配，导致训练效果不佳，并且还有一词多义的情况出现，信息量也不多。

于是在分类任务中采用了提示词模版 (Prompt Template)，把一个类别变成一句话，这样更匹配。

```
prompts = [  
    "a photo of the number {}",  
    "a hand drawn digit {}",  
    "the number {}",  
    "a handwritten number {}",  
    "digit {}",  
    'a white handwritten digit "{}" on black background',  
    'a grayscale image of the digit "{}"',  
    "handwritten {}",  
    "the digit {} hand written",  
]
```

图1.5 我在代码实践环节所采用的提示词模版

1.6 Zero-shot 零样本迁移

CLIP 这篇论文主要强调了模型在 zero-shot 方面的应用，在完全没有微调过的情况下，在某些数据集上的效果是要好于某些有监督、在该数据集上训练过的模型。

然而，CLIP 的强泛化能力也不仅仅体现在不需要微调上面，以前的模型，在做分类任务的时候，训练出的类别是固定的，识别出的类别也是固定的。

但，CLIP 不是这样的，它可以识别出全新的类别。

🔥 让我们来举个例子：

比如给书本做分类，人工设定了几个类别，数学书，英语书，语文书，那么传统模型就只能预测这几种类别的概率，然而 CLIP 不仅能预测出是数学书，还能预测出是高等数学课本，而不是线性代数课本。

1.7 对预测方法的补充说明

(2) Create dataset classifier from label text

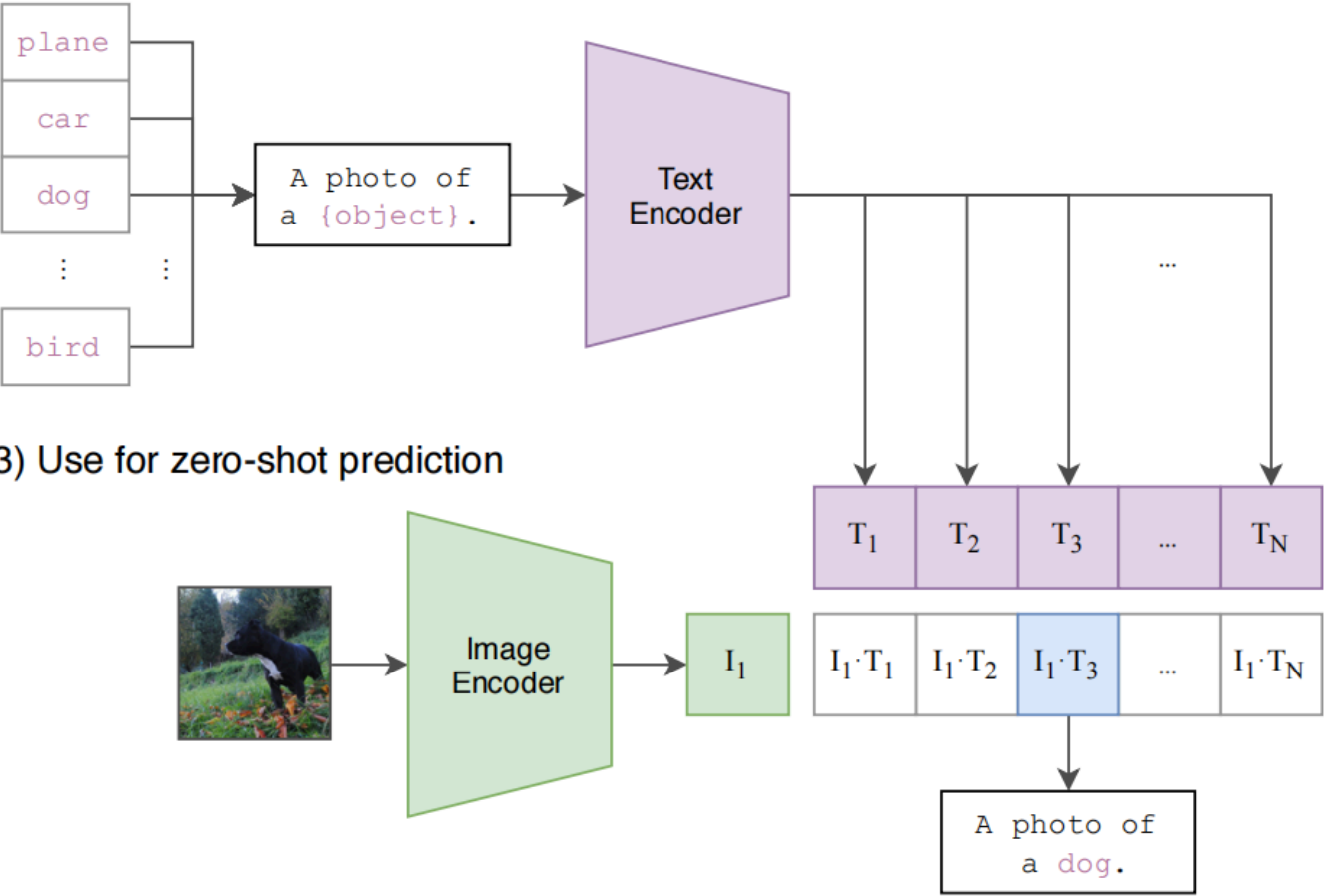


图1.6 预测流程

将待预测的类别输入 Prompt Template，之后模型会对整句话做预测，这 N 句话输入 Text Encoder，得到 N 个向量，将待预测图片输入 Image Encoder 得到一个向量，做余弦相似度，最终输出一个相似度最高的结果 (或者是每个类的预测概率)。

2.CLIP 的局限性

2.1 训练成本很高

如果继续扩大数据集，CLIP 的表现还能更好，但是这会需要巨大的训练资源。

2.2 在一些特定数据集上表现不佳

由于 CLIP 做 zero-shot，如果数据集里的图像比较少见 (例如卫星图像)，那么 CLIP 的精度就不会很高，比起那些有监督训练的模型精度很一般。

2.3 对复杂语义理解困难

例如对图片中的物体进行计数。

2.4 文章中的结果是调整过模型参数才得到的

作者坦言使用了 ImageNet 数据集作为主导，并非真正的 zero-shot。

2.5 数据集未清洗

带有社会偏见。

3.实践环节

由于我对新知识更加渴望，这次仅做了简单的实践，验证为主。

3.1 验证 CLIP 的类别是开放性的

比较显然的一点是在测试脚本中可以自己修改类别和 Prompt Template，开放性显而易见。

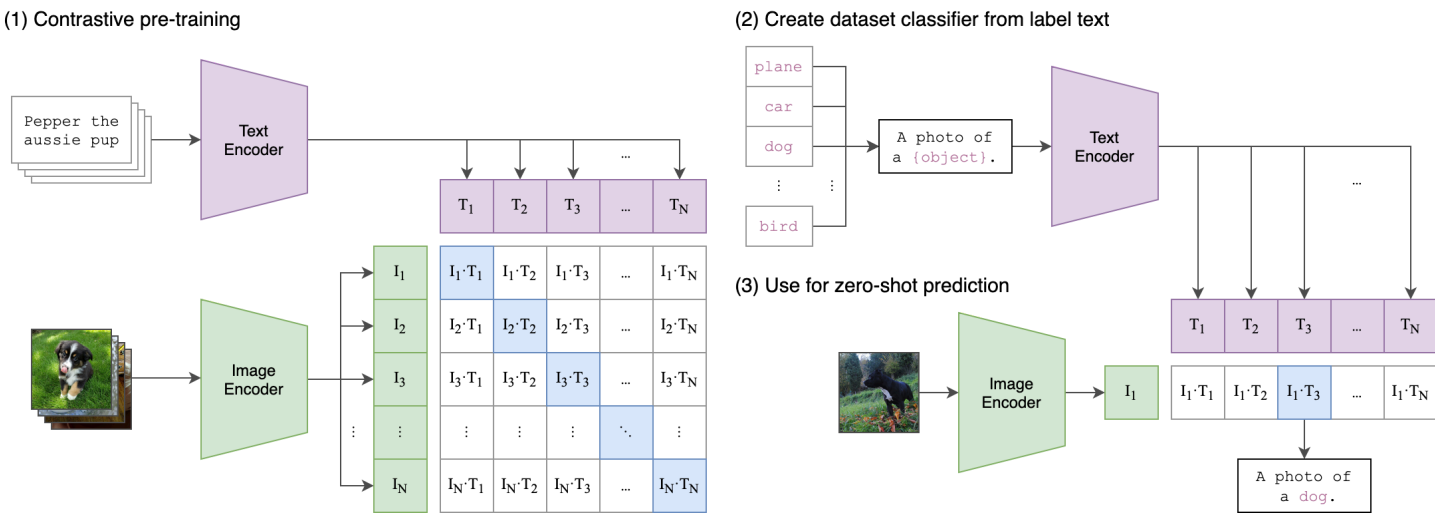


图3.1 测试图片

```
模型：ViT-B/32 | 设备：cuda

图像：CLIP.png
-----
a diagram                -> 0.0124 (1.24%)
a dog                    -> 0.0001 (0.01%)
a cat                    -> 0.0000 (0.00%)
a photo of CLIP          -> 0.0005 (0.05%)
a neural network architecture -> 0.9868 (98.68%)
-----
预测结果：a neural network architecture
```

图3.2 测试结果

可以看出，a neural network architecture 这样较复杂的类别可以识别出来。

3.2 验证 CLIP 在某些数据集上的局限性

使用数据集 MNIST，现在几乎随便一个网络都能在上面达到极高的精确度。

```
模型：ViT-B/32 | 设备：cuda
测试图片数：10000
正确预测：3781
准确率：37.81%

各类别准确率：
数字 0: 99.29% (980 张)
数字 1: 0.00% (1135 张)
数字 2: 0.00% (1032 张)
数字 3: 54.16% (1010 张)
数字 4: 22.51% (982 张)
数字 5: 81.39% (892 张)
数字 6: 15.34% (958 张)
数字 7: 89.88% (1028 张)
数字 8: 24.44% (974 张)
数字 9: 0.50% (1009 张)
```

图3.3 测试结果

CLIP 的结果非常差，zero-shot 的结果很不理想，猜测原因可能有多种，其中之一是类似于 MNIST 的图片-文本对在预训练时候根本没有，所以识别不出某些数字。

预测脚本的代码

```

1  """用CLIP在MNIST测试集全部图片上做零样本分类"""
2  import torch
3  import clip
4  from torch.utils.data import DataLoader
5  from torchvision.datasets import MNIST
6  from tqdm import tqdm
7
8  def main():
9      device = "cuda" if torch.cuda.is_available() else "cpu"
10     batch_size = 256
11
12     # 1. 加载模型
13     model, preprocess = clip.load("ViT-B/32", device=device)
14
15     # 2. 加载MNIST测试集全部图片
16     dataset = MNIST(root="/tmp/mnist_data", train=False, download=True,
17                     transform=preprocess)
18     loader = DataLoader(dataset, batch_size=batch_size, shuffle=False,
19                         num_workers=4)
19
20     # 3. 多模板prompt ensemble
21     prompts = [
22         "a photo of the number {}",
23         "a hand drawn digit {}",
24         "the number {}",
25         "a handwritten number {}",
26         "digit {}",
27         'a white handwritten digit "{}" on black background',
28         'a grayscale image of the digit "{}"',
29         "handwritten {}",
30         "the digit {} hand written",
31     ]
32     num_classes = 10
33
34     # 累积每个prompt模板在各batch上的预测概率
35     all_ensemble_probs = []
36     with torch.no_grad():
37         for prompt_template in tqdm(prompts, desc="prompt ensemble"):
38             texts = [prompt_template.format(i) for i in range(num_classes)]
39             text_tokens = clip.tokenize(texts).to(device)
40             text_features = model.encode_text(text_tokens)
41             text_features = text_features / text_features.norm(dim=-1,
42                         keepdim=True)
42
43             batch_probs = []
44             for imgs, _ in loader:
45                 imgs = imgs.to(device)

```

```

46         image_features = model.encode_image(imgs)
47         image_features = image_features / image_features.norm(dim=-1,
keepdim=True)
48         logit_scale = model.logit_scale.exp()
49         logits = logit_scale * image_features @ text_features.t()
50         probs = logits.softmax(dim=-1).cpu()
51         batch_probs.append(probs)
52         all_ensemble_probs.append(torch.cat(batch_probs))
53
54         # 对多个prompt的预测概率取平均(ensemble)
55         ensemble_probs = torch.stack(all_ensemble_probs).mean(dim=0)
56         predictions = ensemble_probs.argmax(dim=-1)
57         labels = torch.cat([lbls for _, lbls in loader])
58
59         # 4. 计算准确率
60         correct = (predictions == labels).sum().item()
61         accuracy = correct / len(labels) * 100
62
63         print(f"\n模型: ViT-B/32 | 设备: {device}")
64         print(f"测试图片数: {len(labels)}")
65         print(f"正确预测: {correct}")
66         print(f"准确率: {accuracy:.2f}%")
67
68         # 5. 打印每类的准确率
69         print("\n各类别准确率:")
70         for digit in range(num_classes):
71             mask = labels == digit
72             if mask.sum() > 0:
73                 cls_acc = (predictions[mask] == digit).sum().item() /
mask.sum().item() * 100
74                 print(f" 数字 {digit}: {cls_acc:.2f}% ({mask.sum().item()} 张)")
75
76         if __name__ == "__main__":
77             main()
78

```

4.计划

在学长建议下，正在学习 MAE ，受时间限制来不及在工作汇报呈现我学到的内容。